

Series: STIG Hardening Docker Containers & Applications

(Part 4 of 6)

Author: H. Brister Wooley

Published: 2026-05-05

Series: STIG Hardening Docker Containers & Applications (4/6)

Published Link: <https://github.com/hbwooley/deep-thoughts-and-ramblings/blob/main/2026-05-05-stig-hardening-docker-part4.md>

Welcome to Part 4.

Your image is built. It's minimal, non-root, configs have correct ownership per Part 3. Now you run it – and *runtime* is where STIG controls actually get enforced.

The Dockerfile declares intent. `docker-compose.yml` (or your orchestrator) enforces it. This is where `cap_drop`, `read_only`, `security_opt`, `tmpfs`, and `networks` turn “I hope this works” into “the auditor can verify this right now.”

We'll use **real V-IDs from your project's Apache 2.4 STIG** (V-214228 through V-214274) and **Docker STIG** (V-235775+).

The Hardened `docker-compose.yml` (Fully Annotated with V-IDs)

```
version: '3.8'
```

```
services:
  httpd:
    build:
      context: .
      dockerfile: Dockerfile
      target: runtime
    image: httpd-stig:hardened
    container_name: httpd-stig
```

```
    # V-214246 (CAT II): Non-root user (UID 48 per Apache STIG AS24-U1-000360)
```

```
    user: "48:48"
```

```
    # V-214230 (CAT II): Read-only root filesystem (no write access to binaries/config)
```

```

# V-214256 (CAT II): File system protection
read_only: true

# V-214230 (CAT II): tmpfs for writable dirs (no persistent writes to
root FS)
# V-214231 (CAT II): Service monitoring / writable directories
tmpfs:
  - /run/httpd:uid=48,gid=48,mode=750
  - /tmp:uid=48,gid=48,mode=1770
  - /var/cache/httpd:uid=48,gid=48,mode=750

# V-214230 (CAT II): Capability dropping -- only what's needed
# V-214246 (CAT II): Least privilege
cap_drop:
  - ALL
cap_add:
  - NET_BIND_SERVICE      # Bind to ports < 1024 (V-214246: port 80/443)
  - CHOWN                  # For log rotation (V-214231)
  - DAC_OVERRIDE           # For config reload (V-214256)
  - SETGID                 # For worker processes (V-214230)
  - SETUID                 # For worker processes (V-214230)

# V-214230 (CAT II): No new privileges -- blocks setuid/setgid escalation
# V-214230 (CAT II): seccomp profile enforcement
security_opt:
  - no-new-privileges:true
  - seccomp:./seccomp/httpd-profile.json

# V-214230 (CAT II): SELinux confinement (if host supports it)
# security_opt:
#   - label:type:httpd_t
#   - label:level:s0:c123,c456

# V-214246 (CAT II): Network segmentation
networks:
  - frontend
  - backend

# V-214256 (CAT II): Persistent audit logs
# V-214231 (CAT II): Log retention
volumes:
  - ./certs:/etc/pki/tls/certs:ro
  - ./private:/etc/pki/tls/private:ro
  - ./ca:/etc/pki/ca-trust/source/anchors:ro
  - httpd-logs:/var/log/httpd

# V-214231 (CAT II): Health check = continuous verification
# V-214256 (CAT II): Service monitoring
healthcheck:

```

```

    test: ["CMD", "wget", "-q", "--spider", "--no-check-certificate",
"https://localhost/server/api/core/health"]
    interval: 30s
    timeout: 5s
    start_period: 10s
    retries: 3

# V-214230 (CAT II): Resource Limits (prevent DoS)
deploy:
  resources:
    limits:
      cpus: '1.0'
      memory: 512M
    reservations:
      cpus: '0.25'
      memory: 128M

networks:
  frontend:
    driver: bridge
    # V-214246: Ingress only
  backend:
    driver: bridge
    internal: true
    # V-214246: No internet access for backend

volumes:
  httpd-logs:
    driver: local

```

Capability Dropping: The `cap_drop: ALL` Pattern (V-214230, CAT II)

The Apache STIG V-214230 (AS24-U1-000030) requires: “The Apache web server must use cryptography to protect the integrity of remote sessions” – but the broader control is **least privilege and no privilege escalation**.

The Docker STIG V-235777+ requires containers to run with minimal capabilities. Here’s the mapping:

Capability	What It Allows	STIG Control	Why We Drop It
CAP_SYS_ADMIN	Mount, unmount, pivot_root, sysctl	V-214230, V-235777	Full host control – CAT I if present
CAP_SYS_RESOURCE	Override resource limits	V-214228 (DoS)	Resource exhaustion
CAP_NET_RAW	Raw sockets, packet	V-214246 (network)	Network

Capability	What It Allows	STIG Control	Why We Drop It
	crafting		reconnaissance, spoofing
CAP_NET_ADMIN	Interface config, routing, firewall	V-214246	Network manipulation
CAP_DAC_OVERRIDE	Bypass file permissions	V-214256	Read/write any file
CAP_DAC_READ_SEARCH	Read any file, search any dir	V-214256	Info leakage
CAP_SYS_PTRACE	ptrace – debug other processes	V-214230	Credential theft, process injection
CAP_SYS_MODULE	Load kernel modules	V-235777	Rootkit installation
CAP_LINUX_IMMUTABLE	Modify immutable files	V-214256	Tamper with protected files
CAP_IPC_LOCK	Lock memory	V-214228	DoS via memory exhaustion
CAP_LEASE	File leases	V-214228	DoS via file locking
CAP_SYS_CHROOT	chroot()	V-214230	Container escape
CAP_SYS_TTY_CONFIG	TTY configuration	V-214230	Terminal manipulation
CAP_MKNOD	Create device nodes	V-214230	Device access
CAP_AUDIT_WRITE	Write audit logs	V-214231	Log tampering
CAP_AUDIT_CONTROL	Control audit subsystem	V-214231	Audit bypass

We keep only what Apache absolutely needs:

Capability	STIG Justification	V-ID
NET_BIND_SERVICE	Bind to ports 80/443 as non-root	V-214246 (AS24-U1-000360)
CHOWN	Log rotation via logrotate	V-214231
DAC_OVERRIDE	Config reload (graceful restart)	V-214256
SETGID	Worker process group management	V-214230
SETUID	Worker process user management	V-214230

Verification command for every deploy:

```
docker exec httpd-stig capsh --print | grep "Current:"
# Current: =ep
cap_chown,cap_dac_override,cap_setgid,cap_setuid,cap_net_bind_service+ep
# No CAP_SYS_ADMIN, no CAP_NET_RAW, no CAP_SYS_PTRACE, no CAP_SYS_MODULE =
PASS
```

seccomp: The Syscall Firewall (V-214230, V-235777)

Docker's default seccomp profile blocks ~44 syscalls. Our custom profile adds more blocks specific to Apache threat model.

seccomp/httpd-profile.json (annotated with STIG rationale):

```
{
  "defaultAction": "SCMP_ACT_ERRNO",
  "architectures": ["SCMP_ARCH_X86_64", "SCMP_ARCH_X86", "SCMP_ARCH_X32"],
  "syscalls": [
    {
      "names": [
        "read", "write", "open", "close", "stat", "fstat", "lstat",
        "poll", "epoll_wait", "epoll_ctl", "epoll_pwait", "select",
        "pselect6",
        "recvfrom", "sendto", "recvmsg", "sendmsg", "shutdown",
        "socket", "connect", "accept", "accept4", "listen", "bind",
        "getsockname", "getpeername", "setsockopt", "getsockopt",
        "fcntl", "ioctl", "mmap", "munmap", "brk",
        "rt_sigaction", "rt_sigprocmask", "rt_sigreturn",
        "nanosleep", "gettimeofday", "clock_gettime",
        "timerfd_create", "timerfd_settime", "timerfd_gettime",
        "futex", "sched_yield", "getpid", "getppid", "getuid", "geteuid",
        "getgid", "getegid", "gettid", "getrlimit", "setrlimit",
        "getrusage", "times", "uname", "sysinfo",
        "exit", "exit_group", "wait4", "waitid", "prlimit64",
        "mprotect", "madvise", "mincore", "mlock", "mlock2", "mlock2_how", "mlock2_how2", "mlock2_how3", "mlock2_how4", "mlock2_how5", "mlock2_how6", "mlock2_how7", "mlock2_how8", "mlock2_how9", "mlock2_how10", "mlock2_how11", "mlock2_how12", "mlock2_how13", "mlock2_how14", "mlock2_how15", "mlock2_how16", "mlock2_how17", "mlock2_how18", "mlock2_how19", "mlock2_how20", "mlock2_how21", "mlock2_how22", "mlock2_how23", "mlock2_how24", "mlock2_how25", "mlock2_how26", "mlock2_how27", "mlock2_how28", "mlock2_how29", "mlock2_how30", "mlock2_how31", "mlock2_how32", "mlock2_how33", "mlock2_how34", "mlock2_how35", "mlock2_how36", "mlock2_how37", "mlock2_how38", "mlock2_how39", "mlock2_how40", "mlock2_how41", "mlock2_how42", "mlock2_how43", "mlock2_how44", "mlock2_how45", "mlock2_how46", "mlock2_how47", "mlock2_how48", "mlock2_how49", "mlock2_how50", "mlock2_how51", "mlock2_how52", "mlock2_how53", "mlock2_how54", "mlock2_how55", "mlock2_how56", "mlock2_how57", "mlock2_how58", "mlock2_how59", "mlock2_how60", "mlock2_how61", "mlock2_how62", "mlock2_how63", "mlock2_how64", "mlock2_how65", "mlock2_how66", "mlock2_how67", "mlock2_how68", "mlock2_how69", "mlock2_how70", "mlock2_how71", "mlock2_how72", "mlock2_how73", "mlock2_how74", "mlock2_how75", "mlock2_how76", "mlock2_how77", "mlock2_how78", "mlock2_how79", "mlock2_how80", "mlock2_how81", "mlock2_how82", "mlock2_how83", "mlock2_how84", "mlock2_how85", "mlock2_how86", "mlock2_how87", "mlock2_how88", "mlock2_how89", "mlock2_how90", "mlock2_how91", "mlock2_how92", "mlock2_how93", "mlock2_how94", "mlock2_how95", "mlock2_how96", "mlock2_how97", "mlock2_how98", "mlock2_how99", "mlock2_how100", "mlock2_how101", "mlock2_how102", "mlock2_how103", "mlock2_how104", "mlock2_how105", "mlock2_how106", "mlock2_how107", "mlock2_how108", "mlock2_how109", "mlock2_how110", "mlock2_how111", "mlock2_how112", "mlock2_how113", "mlock2_how114", "mlock2_how115", "mlock2_how116", "mlock2_how117", "mlock2_how118", "mlock2_how119", "mlock2_how120", "mlock2_how121", "mlock2_how122", "mlock2_how123", "mlock2_how124", "mlock2_how125", "mlock2_how126", "mlock2_how127", "mlock2_how128", "mlock2_how129", "mlock2_how130", "mlock2_how131", "mlock2_how132", "mlock2_how133", "mlock2_how134", "mlock2_how135", "mlock2_how136", "mlock2_how137", "mlock2_how138", "mlock2_how139", "mlock2_how140", "mlock2_how141", "mlock2_how142", "mlock2_how143", "mlock2_how144", "mlock2_how145", "mlock2_how146", "mlock2_how147", "mlock2_how148", "mlock2_how149", "mlock2_how150", "mlock2_how151", "mlock2_how152", "mlock2_how153", "mlock2_how154", "mlock2_how155", "mlock2_how156", "mlock2_how157", "mlock2_how158", "mlock2_how159", "mlock2_how160", "mlock2_how161", "mlock2_how162", "mlock2_how163", "mlock2_how164", "mlock2_how165", "mlock2_how166", "mlock2_how167", "mlock2_how168", "mlock2_how169", "mlock2_how170", "mlock2_how171", "mlock2_how172", "mlock2_how173", "mlock2_how174", "mlock2_how175", "mlock2_how176", "mlock2_how177", "mlock2_how178", "mlock2_how179", "mlock2_how180", "mlock2_how181", "mlock2_how182", "mlock2_how183", "mlock2_how184", "mlock2_how185", "mlock2_how186", "mlock2_how187", "mlock2_how188", "mlock2_how189", "mlock2_how190", "mlock2_how191", "mlock2_how192", "mlock2_how193", "mlock2_how194", "mlock2_how195", "mlock2_how196", "mlock2_how197", "mlock2_how198", "mlock2_how199", "mlock2_how200", "mlock2_how201", "mlock2_how202", "mlock2_how203", "mlock2_how204", "mlock2_how205", "mlock2_how206", "mlock2_how207", "mlock2_how208", "mlock2_how209", "mlock2_how210", "mlock2_how211", "mlock2_how212", "mlock2_how213", "mlock2_how214", "mlock2_how215", "mlock2_how216", "mlock2_how217", "mlock2_how218", "mlock2_how219", "mlock2_how220", "mlock2_how221", "mlock2_how222", "mlock2_how223", "mlock2_how224", "mlock2_how225", "mlock2_how226", "mlock2_how227", "mlock2_how228", "mlock2_how229", "mlock2_how230", "mlock2_how231", "mlock2_how232", "mlock2_how233", "mlock2_how234", "mlock2_how235", "mlock2_how236", "mlock2_how237", "mlock2_how238", "mlock2_how239", "mlock2_how240", "mlock2_how241", "mlock2_how242", "mlock2_how243", "mlock2_how244", "mlock2_how245", "mlock2_how246", "mlock2_how247", "mlock2_how248", "mlock2_how249", "mlock2_how250", "mlock2_how251", "mlock2_how252", "mlock2_how253", "mlock2_how254", "mlock2_how255", "mlock2_how256", "mlock2_how257", "mlock2_how258", "mlock2_how259", "mlock2_how260", "mlock2_how261", "mlock2_how262", "mlock2_how263", "mlock2_how264", "mlock2_how265", "mlock2_how266", "mlock2_how267", "mlock2_how268", "mlock2_how269", "mlock2_how270", "mlock2_how271", "mlock2_how272", "mlock2_how273", "mlock2_how274", "mlock2_how275", "mlock2_how276", "mlock2_how277", "mlock2_how278", "mlock2_how279", "mlock2_how280", "mlock2_how281", "mlock2_how282", "mlock2_how283", "mlock2_how284", "mlock2_how285", "mlock2_how286", "mlock2_how287", "mlock2_how288", "mlock2_how289", "mlock2_how290", "mlock2_how291", "mlock2_how292", "mlock2_how293", "mlock2_how294", "mlock2_how295", "mlock2_how296", "mlock2_how297", "mlock2_how298", "mlock2_how299", "mlock2_how300", "mlock2_how301", "mlock2_how302", "mlock2_how303", "mlock2_how304", "mlock2_how305", "mlock2_how306", "mlock2_how307", "mlock2_how308", "mlock2_how309", "mlock2_how310", "mlock2_how311", "mlock2_how312", "mlock2_how313", "mlock2_how314", "mlock2_how315", "mlock2_how316", "mlock2_how317", "mlock2_how318", "mlock2_how319", "mlock2_how320", "mlock2_how321", "mlock2_how322", "mlock2_how323", "mlock2_how324", "mlock2_how325", "mlock2_how326", "mlock2_how327", "mlock2_how328", "mlock2_how329", "mlock2_how330", "mlock2_how331", "mlock2_how332", "mlock2_how333", "mlock2_how334", "mlock2_how335", "mlock2_how336", "mlock2_how337", "mlock2_how338", "mlock2_how339", "mlock2_how340", "mlock2_how341", "mlock2_how342", "mlock2_how343", "mlock2_how344", "mlock2_how345", "mlock2_how346", "mlock2_how347", "mlock2_how348", "mlock2_how349", "mlock2_how350", "mlock2_how351", "mlock2_how352", "mlock2_how353", "mlock2_how354", "mlock2_how355", "mlock2_how356", "mlock2_how357", "mlock2_how358", "mlock2_how359", "mlock2_how360", "mlock2_how361", "mlock2_how362", "mlock2_how363", "mlock2_how364", "mlock2_how365", "mlock2_how366", "mlock2_how367", "mlock2_how368", "mlock2_how369", "mlock2_how370", "mlock2_how371", "mlock2_how372", "mlock2_how373", "mlock2_how374", "mlock2_how375", "mlock2_how376", "mlock2_how377", "mlock2_how378", "mlock2_how379", "mlock2_how380", "mlock2_how381", "mlock2_how382", "mlock2_how383", "mlock
```

```

        "sched_getaffinity", "sched_setaffinity", "sched_getparam",
"sched_setparam",
        "sched_getscheduler", "sched_setscheduler", "sched_rr_get_interval",
        "mlock2", "copy_file_range", "statx", "io_pgetevents", "io_submit",
        "io_setup", "io_destroy", "io_getevents", "io_cancel",
        "pselect6", "ppoll", "readv", "writev", "preadv", "pwritev",
        "sendfile", "sendfile64", "fallocate", "fallocate64",
        "fadvise64", "fadvise64_64", "sync", "fsync", "fdatasync",
        "sync_file_range", "vmsplice", "splice", "tee",
        "recvmmsg", "sendmmsg", "process_vm_readv", "process_vm_writev",
        "kcmp", "finit_module", "getrandom"
    ],
    "action": "SCMP_ACT_ALLOW"
}
]
}

```

Key BLOCKED syscalls (defaultAction = SCMP_ACT_ERRNO):

Blocked Syscall	STIG Rationale	V-ID
ptrace	Process injection, credential theft	V-214230
process_vm_readv/writev	Cross-process memory access	V-214230
kcmp	Container escape (kernel comparison)	V-214230, V-235777
userfaultfd	Memory manipulation attacks	V-214230
bpf	eBPF program loading (can escape)	V-214230, V-235777
clone/clone3/fork/vfork	Process creation (Apache manages workers)	V-214230
execve/execveat	Binary execution (no shell = no exec)	V-214230
mount/umount/pivot_root	Filesystem manipulation	V-214230, V-214256
init_module/finit_module/delete_module	Kernel module loading	V-235777
kexec_load	Kernel replacement	V-235777
perf_event_open	Performance monitoring (info leak)	V-214230
seccomp	Modify seccomp filter	V-214230
landlock_create_ruleset	LSM policy manipulation	V-214230

Verification:

```

# Check seccomp is active
docker exec httpd-stig sh -c 'cat /proc/self/status | grep Seccomp'

```

```
# Seccomp: 2 (2 = filter active, 0 = disabled, 1 = strict)

# Verify specific syscall blocked
docker exec httpd-stig sh -c 'python3 -c "import ctypes;
ctypes.CDLL(None).ptrace(0,0,0,0)"' 2>&1
# Should fail with: Function not implemented / Operation not permitted
```

SELinux: The Mandatory Access Control Layer (V-214230, V-214256)

If your host runs SELinux (RHEL 9, Rocky 9, AlmaLinux 9, Fedora), use it. It's free defense-in-depth.

```
security_opt:
- label:type:httpd_t
- label:level:s0:c123,c456
```

Label	STIG Purpose	V-ID
type:httpd_t	Confines to httpd_t domain – only read httpd_config_t, write httpd_log_t, httpd_var_run_t, httpd_cache_t	V-214230, V-214256
level:s0:c123,c456	MCS isolation – separates this container from others	V-214230, V-214246

STIG Controls Satisfied: - **V-214230:** Mandatory access control prevents lateral movement - **V-214256:** File access controlled by policy, not just DAC - **V-214246:** Network access controlled by policy (httpd_t can only connect to http_port_t)

Verification:

```
docker exec httpd-stig sh -c 'cat /proc/self/attr/current'
# system_u:system_r:httpd_t:s0:c123,c456

# Verify file contexts
docker exec httpd-stig ls -Z /usr/local/apache2/conf/
# system_u:object_r:httpd_config_t:s0 httpd.conf
# system_u:object_r:httpd_config_t:s0 ssl.conf
```

The One Setting That Turns CAT II into CAT I

```
privileged: true
```

If you see this in a compose file, **stop**. A privileged container: - Has ALL capabilities (including CAP_SYS_ADMIN, CAP_SYS_MODULE) - No seccomp filter (or unconfined) - No SELinux confinement - Access to all host devices (/dev/*) - Can load kernel modules - Can mount host filesystems - Can modify kernel parameters via /proc/sys

That’s not a CAT II finding (severity medium). That’s a **CAT I finding (severity critical)** – V-235777 “The system allows unrestricted privileged access.”

Never use privileged: true. If your app “needs” it, fix the app. Use cap_add for the specific capability, device: for the specific device, tmpfs: for the specific mount.

Network Segmentation: The V-214246 Enforcement

```
networks:
  frontend:
    driver: bridge
  backend:
    driver: bridge
    internal: true # No internet access, no host access
```

Network	STIG Purpose	V-ID
frontend	Ingress only – receives TLS-terminated traffic	V-214246 (AS24-U1-000360)
backend	internal: true – no outbound, no host access	V-214246, V-214230

Verification:

```
# Backend container cannot reach internet
docker exec httpd-stig wget -q --spider http://google.com
# Should fail: Network is unreachable

# Backend cannot reach host metadata
docker exec httpd-stig wget -q --spider http://169.254.169.254
# Should fail
```

What This Means for Your Strategy

Runtime hardening is where STIG becomes *operational*. The image is static; the runtime is dynamic. The compose file is your runtime contract.

Every setting in that compose file should have a V-ID comment. Every cap_add should have a justification. Every tmpfs mount should reference the STIG control it satisfies.

And the verification commands? They’re your **continuous audit**. Run them in CI. Run them in a nightly job. Run them when the auditor shows up.

What's Coming Next

Part 5: Application-layer STIG. The controls your *code* must implement – input validation, output encoding, session management, audit logging, cryptographic standards. The STIG controls that no container setting can fix.

Plus the one application finding that fails 90% of audits – and how to fix it in 10 lines of code.

[Read Part 5 ->](#)

Appendix: CompTIA Security+ SY0-701 CEU Mapping

This appendix maps each technique in this post to CompTIA Security+ exam objectives for CEU submission.

Technique Implemented	Security+ Domain	SY0-701 Objective	Rationale
cap_drop: ALL + selective cap_add	Domain 3: Security Architecture	3.2: Implement secure system design	Least privilege capabilities, removes dangerous defaults
read_only: true root filesystem	Domain 3: Security Architecture	3.2: Implement secure system design	Immutable container runtime prevents tampering
no-new-privileges:true	Domain 3: Security Architecture	3.2: Implement secure system design	Prevents privilege escalation via setuid binaries
Custom seccomp profile (blocking 44+ syscalls)	Domain 3: Security Architecture	3.2: Implement secure system design	Syscall filtering reduces kernel attack surface
ptrace, execve, clone, mount blocked	Domain 3: Security Architecture	3.2: Implement secure system design	Prevents process injection, binary execution, container escape
SELinux httpd_t confinement	Domain 3: Security Architecture	3.4: Implement secure network architecture	Mandatory access control, domain isolation
MCS labels (s0:c123,c456)	Domain 3: Security Architecture	3.4: Implement secure network architecture	Multi-category

Technique Implemented	Security+ Domain	SY0-701 Objective	Rationale
backend: internal: true network	Domain 3: Security Architecture	3.4: Implement secure network architecture	separation between containers Backend isolation, no internet egress
Resource limits (CPU/memory)	Domain 3: Security Architecture	3.2: Implement secure system design	DoS prevention via resource quotas
Volume-mounted persistent logs	Domain 4: Security Operations	4.1: Apply security monitoring concepts	Audit trail survives container lifecycle
Runtime verification commands in CI/CD	Domain 4: Security Operations	4.5: Implement automation/orchestration	Automated compliance validation

Primary Domain Coverage: Domain 3 (Architecture & Engineering) – ~65% | Domain 4 (Operations) – ~35%

Total Security+ Mapping: >=50% of content maps to SY0-701 objectives